long as possible. The Ss-Tree version, of course, is even more complicated than the original one, it goes without saying!

But one of the aspects where R-trees and Ss-trees overcome k-d trees is that while the latter is guaranteed to be balanced only if initialized on a static dataset, those two can remain balanced even when supporting dynamic datasets, with a large volume of inserts and removals: giving up on delete would therefore mean turning down one of the main reasons we need this data structure.

The first (and easiest) step is, of course, finding the point we would like to delete, or better said, finding the leaf that holds that point. While for `insert` we would only traverse one path to the closest leaf, for `delete` we are back at the search algorithm described in section 10.3.1; however, like for `insert`, we will need to perform some backtracking, and hence rather than calling search, we will have to implement the same traversal in this new method[13].

Once we have found the right leaf `L`, assuming we do find the point `Z` in the tree (otherwise we wouldn't need to perform any change), we have a few possible situations: an easy one, a complicated one, and a seriously complicated one.

1. If the leaf contains more than `m` points, we just delete `Z` from `L`, and update its bounding envelope.
2. Otherwise, after deleting `Z`, `L` will have only `m-1` points, and therefore it would violate one of the Ss-tree's invariants. We have a few options to cope with this: